

ETL/ELT patterns with Apache Airflow®

7 Practical DAG Code Examples

ASTRONOMER

DAG Code Examples

1. ETL pattern DAG using XCom	4
2. ETL pattern DAG using explicit external storage	9
3. ELT pattern DAG using XCom	16
4. ELT pattern DAG using explicit external storage	20
5. ETLT DAG using XCom	25
6. Advanced ETL DAG with data quality checks	30
7. ELT DAG using Dynamic Task Mapping	37

Introduction

When writing data pipelines, regardless of the end use case—whether you provide data for analytics, complex dashboards, or are fine-tuning large language models—your data must first be **extracted (E)** from its source, **transformed (T)** into a target schema, and **loaded (L)** into your central data storage, whether that's a data warehouse or a data lake.

ETL and ELT pipelines form the core foundation of any data architecture, making the ability to design, build, and understand these pipelines essential for every data engineer.

Originally developed in 2014 specifically for ETL and ELT pipelines, [Apache Airflow®](#) has since evolved into the open-source standard for full-stack orchestration of any workflow—without losing sight of its roots in moving and transforming data.

According to the [2023 Airflow survey](#), 95% of current Airflow users rely on it to run ETL and ELT pipelines, powering everything from some of the world's largest organizations to fast-growing startups.

This eBook contains full example DAG patterns for different ETL and ELT scenarios. The code to all these DAGs can also be found in [this GitHub repository](#), which is set up for you to be able to run them locally without needing to configure any connections or additional tools.

While the example DAGs use Postgres as their target system, and MinIO as external storage, they can be adapted to interact with any tool that is accessible through an API. For many tools, pre-built modules exist, see the [Astronomer Registry](#) for a comprehensive list.

helpful hints

This eBook is aimed at intermediate Airflow users. If you are new to Airflow we recommend first completing the [Astronomer Get Started](#) tutorial to get familiar with core Airflow concepts before using the code in this eBook.

[Complete the Tutorial](#)

If you are looking for more guidance on how to use Airflow for ETL and ELT, check out the [Best practices for ELT and ETL pipelines with Apache Airflow®](#) Ebook for all you need to know to write ETL and ELT pipelines.

[Read the eBook](#)

ETL pattern DAG using XCom

This DAG shows an ETL pattern to extract data from the Open-Meteo API, transform it and then load it into a Postgres database. The data is passed through XCom between the extraction and transformation task, as well as between the transformation and loading task, which means this setup would need a **custom XCom backend** in production.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
## Simple ETL DAG loading data from the Open-Meteo API to a Postgres database

This DAG extracts weather data from the Open-Meteo API, transforms it, and
loads it into a Postgres database in an ETL pattern.
It passes the data through XComs.
"""

import os
from datetime import datetime, timedelta

from airflow.decorators import dag, task
from airflow.models.baseoperator import chain
from airflow.models.param import Param
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

# ----- #
# DAG-level variables #
# ----- #

DAG_ID = os.path.basename(__file__).replace(".py", "")
```

```

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../../include/sql/pattern_dags/{DAG_ID}"
)

# ----- #
# DAG definition #
# ----- #

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ETL", "Postgres", "XCom"], # add tags in the UI
    params={
        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)
def etl_xcom():

    # ----- #
    # Task Definitions #
    # ----- #

```

```

# the @task decorator turns any Python function into an Airflow task
# any @task decorated function that is called inside the @dag decorated
# function is automatically added to the DAG.
# if one exists for your use case you can use traditional Airflow operators
# and mix them with @task decorators. Checkout registry.astronomer.io for available
operators

# see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
@task

# see: https://www.astronomer.io/docs/learn/what-is-an-operator for information
about traditional operators

```

```

_create_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
)

```

```

@task
def extract(**context):
    """
    Extract data from the Open-Meteo API
    Returns:
        dict: The full API response
    """
    import requests

    url = os.getenv("WEATHER_API_URL")

    coordinates = context["params"]["coordinates"]
    latitude = coordinates["latitude"]
    longitude = coordinates["longitude"]

    url = url.format(latitude=latitude, longitude=longitude)

    response = requests.get(url)

    return response.json()

```

```

@task
def transform(api_response: dict, **context) → dict:
    """
    Transform the data

```

Args:

api_response (dict): The full API response

Returns:

dict: The transformed data

"""

```
time = api_response["hourly"]["time"]
```

```
dag_run_timestamp = context["ts"]
```

```
transformed_data = {
```

```
    "temperature_2m": api_response["hourly"]["temperature_2m"],
```

```
    "relative_humidity_2m": api_response["hourly"]["relative_humidity_2m"],
```

```
    "precipitation_probability": api_response["hourly"]
```

```
        "precipitation_probability"
```

```
],
```

```
    "timestamp": time,
```

```
    "date": [
```

```
        datetime.strptime(x, "%Y-%m-%dT%H:%M").date().strftime("%Y-%m-%d")
```

```
        for x in time
```

```
],
```

```
    "day": [datetime.strptime(x, "%Y-%m-%dT%H:%M").day for x in time],
```

```
    "month": [datetime.strptime(x, "%Y-%m-%dT%H:%M").month for x in time],
```

```
    "year": [datetime.strptime(x, "%Y-%m-%dT%H:%M").year for x in time],
```

```
    "last_updated": [dag_run_timestamp for i in range(len(time))],
```

```
    "latitude": [api_response["latitude"] for i in range(len(time))],
```

```
    "longitude": [api_response["longitude"] for i in range(len(time))],
```

```
}
```

```
return transformed_data
```

@task

```
def load(transformed_data: dict):
```

"""

Load the data to the destination without using a temporary CSV file.

Args:

transformed_data (dict): The transformed data

"""

```
import csv
```

```
import io
```

```
from airflow.providers.postgres.hooks.postgres import PostgresHook
```

```
hook = PostgresHook(postgres_conn_id=_POSTGRES_CONN_ID)
```

```
csv_buffer = io.StringIO()
```

```

writer = csv.writer(csv_buffer)
writer.writerow(transformed_data.keys())
rows = zip(*transformed_data.values())
writer.writerows(rows)

csv_buffer.seek(0)

with open(f"{_SQL_DIR}/copy_insert.sql") as f:
    sql = f.read()
    sql = sql.replace("{schema}", _POSTGRES_SCHEMA)
    sql = sql.replace("{table}", _POSTGRES_TRANSFORMED_TABLE)

conn = hook.get_conn()
cursor = conn.cursor()
cursor.copy_expert(sql=sql, file=csv_buffer)
conn.commit()
cursor.close()
conn.close()

_extract = extract()
_transform = transform(api_response=_extract)
_load = load(transformed_data=_transform)
chain(_transform, _load)
chain(_create_table_if_not_exists, _load)

etl_xcom()

```


ETL pattern DAG using explicit external storage

This DAG accomplishes the same task, extracting data from the Open-Meteo API, transforming it and loading it to a Postgres database, but it does not pass any data through XCom, instead, MinIO is used as explicit external storage.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
```

```
## Simple ETL DAG loading data from the Open-Meteo API to a Postgres database
```

```
This DAG extracts weather data from the Open-Meteo API, transforms it, and  
loads it into a Postgres database in an ETL pattern.
```

```
It uses S3 as an intermediary storage.
```

```
"""
```

```
import json
```

```
import os
```

```
from datetime import datetime, timedelta
```

```
import requests
```

```
from airflow.decorators import dag, task, task_group
```

```
from airflow.models.baseoperator import chain
```

```
from airflow.models.param import Param
```

```
from airflow.providers.amazon.aws.hooks.s3 import S3Hook
```

```
from airflow.providers.amazon.aws.operators.s3 import S3CreateBucketOperator
```

```
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
```

```

# ----- #
# DAG-level variables #
# ----- #

DAG_ID = os.path.basename(__file__).replace(".py", "")

_AWS_CONN_ID = os.getenv("MINIO_CONN_ID", "minio_local")
_S3_BUCKET = os.getenv("S3_BUCKET", "open-meteo-etl")

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../..../include/sql/pattern_dags/{DAG_ID}"
)

_EXTRACT_TASK_ID = "extract"
_TRANSFORM_TASK_ID = "transform"

# ----- #
# DAG definition #
# ----- #

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
    flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
    runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
    custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ETL", "Postgres", "Intermediary Storage"], # add tags in the UI
    params={

```

```

        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)
def etl_intermediary_storage():

    # ----- #
    # Task Definitions #
    # ----- #

    # the @task decorator turns any Python function into an Airflow task
    # any @task decorated function that is called inside the @dag decorated
    # function is automatically added to the DAG.
    # if one exists for your use case you can use traditional Airflow operators
    # and mix them with @task decorators. Checkout registry.astronomer.io for available
operators
    # see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
@task
    # see: https://www.astronomer.io/docs/learn/what-is-an-operator for information
about traditional operators

    @task_group
    def tool_setup():

        _create_table_if_not_exists = SQLExecuteQueryOperator(
            task_id="create_table_if_not_exists",
            conn_id=_POSTGRES_CONN_ID,
            database=_POSTGRES_DATABASE,
            sql="create_table_if_not_exists.sql",
            params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
        )

        _create_bucket_if_not_exists = S3CreateBucketOperator(
            task_id="create_bucket_if_not_exists",
            bucket_name=_S3_BUCKET,
            aws_conn_id=_AWS_CONN_ID,
        )

        return _create_table_if_not_exists, _create_bucket_if_not_exists

    _tool_setup = tool_setup()

    @task(task_id=_EXTRACT_TASK_ID)
    def extract(**context):

```



```

"""
Extract data from the Open-Meteo API
Returns:
    dict: The full API response
"""

url = os.getenv("WEATHER_API_URL")

coordinates = context["params"]["coordinates"]
latitude = coordinates["latitude"]
longitude = coordinates["longitude"]
dag_run_timestamp = context["ts"]
dag_id = context["dag"].dag_id
task_id = context["task"].task_id

url = url.format(latitude=latitude, longitude=longitude)

response = requests.get(url).json()

response_bytes = json.dumps(response).encode("utf-8")

# Save the data to S3
hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
hook.load_bytes(
    bytes_data=response_bytes,
    key=f"{dag_id}/{task_id}/{dag_run_timestamp}.json",
    bucket_name=_S3_BUCKET,
    replace=True,
)

@task(task_id=_TRANSFORM_TASK_ID)
def transform(**context):
    """
    Transform the data
    Args:
        api_response (dict): The full API response
    Returns:
        dict: The transformed data
    """

    dag_run_timestamp = context["ts"]
    dag_id = context["dag"].dag_id
    upstream_task_id = _EXTRACT_TASK_ID
    task_id = context["task"].task_id

```

```

# Load the data from S3
hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
response = hook.read_key(
    key=f"{dag_id}/{upstream_task_id}/{dag_run_timestamp}.json",
    bucket_name=_S3_BUCKET,
)
api_response = json.loads(response)

time = api_response["hourly"]["time"]

transformed_data = {
    "temperature_2m": api_response["hourly"]["temperature_2m"],
    "relative_humidity_2m": api_response["hourly"]["relative_humidity_2m"],
    "precipitation_probability": api_response["hourly"][
        "precipitation_probability"
    ],
    "timestamp": time,
    "date": [
        datetime.strptime(x, "%Y-%m-%dT%H:%M").date().strftime("%Y-%m-%d")
        for x in time
    ],
    "day": [datetime.strptime(x, "%Y-%m-%dT%H:%M").day for x in time],
    "month": [datetime.strptime(x, "%Y-%m-%dT%H:%M").month for x in time],
    "year": [datetime.strptime(x, "%Y-%m-%dT%H:%M").year for x in time],
    "last_updated": [dag_run_timestamp for i in range(len(time))],
    "latitude": [api_response["latitude"] for i in range(len(time))],
    "longitude": [api_response["longitude"] for i in range(len(time))],
}

transformed_data_bytes = json.dumps(transformed_data).encode("utf-8")

# Save the data to S3
hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
hook.load_bytes(
    bytes_data=transformed_data_bytes,
    key=f"{dag_id}/{task_id}/{dag_run_timestamp}.json",
    bucket_name=_S3_BUCKET,
    replace=True,
)

@task
def load(**context):

```

```
"""
```

Load the data to the destination without using a temporary CSV file.

Args:

transformed_data (dict): The transformed data

```
"""
```

```
import csv
```

```
import io
```

```
from airflow.providers.postgres.hooks.postgres import PostgresHook
```

```
dag_run_timestamp = context["ts"]
```

```
dag_id = context["dag"].dag_id
```

```
upstream_task_id = _TRANSFORM_TASK_ID
```

```
# Load the data from S3
```

```
hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
```

```
response = hook.read_key(
```

```
    key=f"{dag_id}/{upstream_task_id}/{dag_run_timestamp}.json",
```

```
    bucket_name=_S3_BUCKET,
```

```
)
```

```
api_response = json.loads(response)
```

```
# Load the data to Postgres
```

```
hook = PostgresHook(postgres_conn_id=_POSTGRES_CONN_ID)
```

```
csv_buffer = io.StringIO()
```

```
writer = csv.writer(csv_buffer)
```

```
writer.writerow(api_response.keys())
```

```
rows = zip(*api_response.values())
```

```
writer.writerows(rows)
```

```
csv_buffer.seek(0)
```

```
with open(f"{_SQL_DIR}/copy_insert.sql") as f:
```

```
    sql = f.read()
```

```
sql = sql.replace("{schema}", _POSTGRES_SCHEMA)
```

```
sql = sql.replace("{table}", _POSTGRES_TRANSFORMED_TABLE)
```

```
conn = hook.get_conn()
```

```
cursor = conn.cursor()
```

```
cursor.copy_expert(sql=sql, file=csv_buffer)
```

```
conn.commit()
```

```
        cursor.close()
        conn.close()

    _extract = extract()
    _transform = transform()
    _load = load()
    chain(_extract, _transform, _load)
    chain(_tool_setup[0], _load)
    chain(_tool_setup[1], _extract)

    etl_intermediary_storage()
```


ELT pattern DAG using XCom

In this example DAG, the full data extracted from the Open-Meteo API is loaded into a Postgres database and all transformation happens inside the Postgres database using SQL. The data is passed through XCom between the extraction and loading task, which means this setup would need a [custom XCom backend](#) in production.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
```

```
## Simple ELT DAG loading data from the Open-Meteo API to a Postgres database
```

```
This DAG extracts weather data from the Open-Meteo API,  
loads it into a Postgres database and transforms it, using an ELT pattern.  
It passes the data through XComs between extract and transform.
```

```
"""
```

```
import os
```

```
import json
```

```
from datetime import datetime, timedelta
```

```
from airflow.decorators import dag, task
```

```
from airflow.models.baseoperator import chain
```

```
from airflow.models.param import Param
```

```
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
```

```

# ----- #
# DAG-level variables #
# ----- #

DAG_ID = os.path.basename(__file__).replace(".py", "")

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_IN_TABLE = os.getenv("POSTGRES_WEATHER_TABLE_IN", f"in_weather_data_{DAG_ID}")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../../include/sql/pattern_dags/{DAG_ID}"
)

# ----- #
# DAG definition #
# ----- #

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
    flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
    runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
    custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ELT", "Postgres", "XCom"], # add tags in the UI
    params={
        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
    astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)

```

```
def elt_xcom():

    # ----- #
    # Task Definitions #
    # ----- #
    # the @task decorator turns any Python function into an Airflow task
    # any @task decorated function that is called inside the @dag decorated
    # function is automatically added to the DAG.
    # if one exists for your use case you can use traditional Airflow operators
    # and mix them with @task decorators. Checkout registry.astronomer.io for available
operators
    # see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
@task
    # see: https://www.astronomer.io/docs/learn/what-is-an-operator for information
about traditional operators

_create_in_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_in_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_in_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_IN_TABLE},
)

_create_model_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_model_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_model_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
)

@task
def extract(**context):
    """
    Extract data from the Open-Meteo API
    Returns:
        dict: The full API response
    """
    import requests

    url = os.getenv("WEATHER_API_URL")

    coordinates = context["params"]["coordinates"]
    latitude = coordinates["latitude"]
    longitude = coordinates["longitude"]
```

```

url = url.format(latitude=latitude, longitude=longitude)

response = requests.get(url)

return response.json()

_extract = extract()

_load = SQLExecuteQueryOperator(
    task_id="load_raw_data",
    conn_id=_POSTGRES_CONN_ID,
    sql="""
INSERT INTO {{ params.schema }}.{{ params.table }} (raw_data)
VALUES ('{{ ti.xcom_pull(task_ids='extract') | tojson }}'::jsonb);
""",
    params={
        "schema": _POSTGRES_SCHEMA,
        "table": _POSTGRES_IN_TABLE,
    },
)

_transform = SQLExecuteQueryOperator(
    task_id="transform_data",
    conn_id=_POSTGRES_CONN_ID,
    sql="transform.sql",
    params={
        "schema": _POSTGRES_SCHEMA,
        "in_table": _POSTGRES_IN_TABLE,
        "out_table": _POSTGRES_TRANSFORMED_TABLE,
    },
)

chain([_create_in_table_if_not_exists, _extract], _load, _transform)
chain(_create_model_table_if_not_exists, _transform)

elt_xcom()

chain([_create_table_if_not_exists, _extract], _load, _transform)

elt_xcom()

```


ELT pattern DAG using explicit external storage

This DAG accomplishes the same as the above DAG, extracting data from the Open-Meteo API, loading the raw data into a Postgres database and then running a transformation on it, but without passing any data through XCom.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
```

```
## Simple ELT DAG loading data from the Open-Meteo API to a Postgres database
```

```
This DAG extracts weather data from the Open-Meteo API,
loads it into a Postgres database and transforms it, using an ELT pattern.
It passes the data through XCom between extract and transform.
"""
```

```
import os
import json
from datetime import datetime, timedelta

from airflow.providers.amazon.aws.hooks.s3 import S3Hook
from airflow.decorators import dag, task
from airflow.models.baseoperator import chain
from airflow.models.param import Param
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
```

```
# ----- #
# DAG-level variables #
# ----- #
```

```
DAG_ID = os.path.basename(__file__).replace(".py", "")
```

```

_AWS_CONN_ID = os.getenv("MINIO_CONN_ID", "minio_local")
_S3_BUCKET = os.getenv("S3_BUCKET", "open-meteo-etl")

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_IN_TABLE = os.getenv("POSTGRES_WEATHER_TABLE_IN", f"in_weather_data_{DAG_ID}")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../include/sql/pattern_dags/{DAG_ID}"
)

_EXTRACT_TASK_ID = "extract"

# ----- #
# DAG definition #
# ----- #

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
    flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
    runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
    custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ELT", "Postgres", "XCom"], # add tags in the UI
    params={
        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
    astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)

```

```
def elt_intermediary_storage():

    # ----- #
    # Task Definitions #
    # ----- #

    # the @task decorator turns any Python function into an Airflow task
    # any @task decorated function that is called inside the @dag decorated
    # function is automatically added to the DAG.
    # if one exists for your use case you can use traditional Airflow operators
    # and mix them with @task decorators. Checkout registry.astronomer.io for available
    operators
    # see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
    @task
    # see: https://www.astronomer.io/docs/learn/what-is-an-operator for information
    about traditional operators

    _create_in_table_if_not_exists = SQLExecuteQueryOperator(
        task_id="create_in_table_if_not_exists",
        conn_id=_POSTGRES_CONN_ID,
        database=_POSTGRES_DATABASE,
        sql="create_in_table_if_not_exists.sql",
        params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_IN_TABLE},
    )

    _create_model_table_if_not_exists = SQLExecuteQueryOperator(
        task_id="create_model_table_if_not_exists",
        conn_id=_POSTGRES_CONN_ID,
        database=_POSTGRES_DATABASE,
        sql="create_model_table_if_not_exists.sql",
        params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
    )

    @task(task_id=_EXTRACT_TASK_ID)
    def extract(**context):
        """
        Extract data from the Open-Meteo API
        Returns:
            dict: The full API response
        """
        import requests

        url = os.getenv("WEATHER_API_URL")

        coordinates = context["params"]["coordinates"]
        latitude = coordinates["latitude"]
```

```

longitude = coordinates["longitude"]
dag_run_timestamp = context["ts"]
dag_id = context["dag"].dag_id
task_id = context["task"].task_id

url = url.format(latitude=latitude, longitude=longitude)

response = requests.get(url).json()

response_bytes = json.dumps(response).encode("utf-8")

# Save the data to S3
hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
hook.load_bytes(
    bytes_data=response_bytes,
    key=f"{dag_id}/{task_id}/{dag_run_timestamp}.json",
    bucket_name=_S3_BUCKET,
    replace=True,
)

_extract = extract()

@task
def load(**context):
    """
    Load the data from S3 to Postgres
    """

    dag_run_timestamp = context["ts"]
    dag_id = context["dag"].dag_id
    upstream_task_id = _EXTRACT_TASK_ID

    print(f"{dag_id}/{upstream_task_id}/{dag_run_timestamp}.json")

    s3_hook = S3Hook(aws_conn_id=_AWS_CONN_ID)
    response = s3_hook.read_key(
        key=f"{dag_id}/{upstream_task_id}/{dag_run_timestamp}.json",
        bucket_name=_S3_BUCKET,
    )
    api_response = json.loads(response)

    postgres_hook = PostgresHook(postgres_conn_id=_POSTGRES_CONN_ID)

    insert_sql = f"""

```



```

INSERT INTO {_POSTGRES_SCHEMA}._{POSTGRES_IN_TABLE} (raw_data)
VALUES (%s::jsonb);
"""

postgres_hook.run(insert_sql, parameters=(json.dumps(api_response),))

_transform = SQLExecuteQueryOperator(
    task_id="transform_data",
    conn_id=_POSTGRES_CONN_ID,
    sql="transform.sql",
    params={
        "schema": _POSTGRES_SCHEMA,
        "in_table": _POSTGRES_IN_TABLE,
        "out_table": _POSTGRES_TRANSFORMED_TABLE,
    },
)

chain([_create_in_table_if_not_exists, _extract], load(), _transform)
chain(_create_model_table_if_not_exists, _transform)

elt_intermediary_storage()

```

ETLT DAG using XCom

In real world scenarios ETL and ELT are often combined to form ETLT, a pattern where both types of transformations happen, once in flight before loading the data and once more after loading the data to a data warehouse.

This example shows the same ETL steps as the [ETL pattern DAG using XCom](#) DAG, with an added additional transformation step that creates a secondary table with aggregated information.

Note that the data is passed through XCom between the extraction and transformation task, as well as between the transformation and loading task, which means this setup would need a [custom XCom backend](#) in production.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
```

```
## Simple ETLT DAG loading data from the Open-Meteo API to a Postgres database
```

```
This DAG extracts weather data from the Open-Meteo API, transforms it, and
loads it into a Postgres database, where a second transformation is applied.
```

```
Note that it uses XComs to pass data between tasks.
```

```
"""
```

```
import os
```

```
from datetime import datetime, timedelta
```

```
from airflow.decorators import dag, task
```

```
from airflow.models.baseoperator import chain
```

```
from airflow.models.param import Param
```

```
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
```

```
# ----- #
```

```
# DAG-level variables #
```

```
# ----- #
```

```

DAG_ID = os.path.basename(__file__).replace(".py", "")

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
_POSTGRES_SECONDARY_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_SECONDARY", f"secondary_weather_data_{DAG_ID}"
)
_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../..../include/sql/pattern_dags/{DAG_ID}"
)

# ----- #
# DAG definition #
# ----- #

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
    flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
    runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
    custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ETL", "Postgres", "XCom"], # add tags in the UI
    params={
        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
    astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)
def etlt():

```

```

# ----- #
# Task Definitions #
# ----- #
# the @task decorator turns any Python function into an Airflow task
# any @task decorated function that is called inside the @dag decorated
# function is automatically added to the DAG.
# if one exists for your use case you can use traditional Airflow operators
# and mix them with @task decorators. Checkout registry.astronomer.io for available
operators
# see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
@task
# see: https://www.astronomer.io/docs/learn/what-is-an-operator for information
about traditional operators

_create_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
)

@task
def extract(**context):
    """
    Extract data from the Open-Meteo API
    Returns:
        dict: The full API response
    """
    import requests

    url = os.getenv("WEATHER_API_URL")

    coordinates = context["params"]["coordinates"]
    latitude = coordinates["latitude"]
    longitude = coordinates["longitude"]

    url = url.format(latitude=latitude, longitude=longitude)

    response = requests.get(url)

    return response.json()

@task
def transform_in_flight(api_response: dict, **context) → dict:

```

```
"""
```

```
Transform the data
```

```
Args:
```

```
    api_response (dict): The full API response
```

```
Returns:
```

```
    dict: The transformed data
```

```
"""
```

```
time = api_response["hourly"]["time"]
```

```
dag_run_timestamp = context["ts"]
```

```
transformed_data = {
```

```
    "temperature_2m": api_response["hourly"]["temperature_2m"],
```

```
    "relative_humidity_2m": api_response["hourly"]["relative_humidity_2m"],
```

```
    "precipitation_probability": api_response["hourly"]
```

```
        "precipitation_probability"
```

```
],
```

```
    "timestamp": time,
```

```
    "date": [
```

```
        datetime.strptime(x, "%Y-%m-%dT%H:%M").date().strftime("%Y-%m-%d")
```

```
        for x in time
```

```
],
```

```
    "day": [datetime.strptime(x, "%Y-%m-%dT%H:%M").day for x in time],
```

```
    "month": [datetime.strptime(x, "%Y-%m-%dT%H:%M").month for x in time],
```

```
    "year": [datetime.strptime(x, "%Y-%m-%dT%H:%M").year for x in time],
```

```
    "last_updated": [dag_run_timestamp for i in range(len(time))],
```

```
    "latitude": [api_response["latitude"] for i in range(len(time))],
```

```
    "longitude": [api_response["longitude"] for i in range(len(time))],
```

```
}
```

```
return transformed_data
```

```
@task
```

```
def load(transformed_data: dict):
```

```
    """
```

```
    Load the data to the destination without using a temporary CSV file.
```

```
Args:
```

```
    transformed_data (dict): The transformed data
```

```
    """
```

```
import csv
```

```
import io
```

```
from airflow.providers.postgres.hooks.postgres import PostgresHook
```

```
hook = PostgresHook(postgres_conn_id=_POSTGRES_CONN_ID)
```



```

csv_buffer = io.StringIO()
writer = csv.writer(csv_buffer)
writer.writerow(transformed_data.keys())
rows = zip(*transformed_data.values())
writer.writerows(rows)

csv_buffer.seek(0)

with open(f"{_SQL_DIR}/copy_insert.sql") as f:
    sql = f.read()
    sql = sql.replace("{schema}", _POSTGRES_SCHEMA)
    sql = sql.replace("{table}", _POSTGRES_TRANSFORMED_TABLE)

conn = hook.get_conn()
cursor = conn.cursor()
cursor.copy_expert(sql=sql, file=csv_buffer)
conn.commit()
cursor.close()
conn.close()

_transform_in_target = SQLExecuteQueryOperator(
    task_id="transform_in_target",
    conn_id=_POSTGRES_CONN_ID,
    sql="transform.sql",
    params={
        "schema": _POSTGRES_SCHEMA,
        "in_table": _POSTGRES_TRANSFORMED_TABLE,
        "out_table": _POSTGRES_SECONDARY_TABLE,
    },
)

_extract = extract()
_transform = transform_in_flight(api_response=_extract)
_load = load(transformed_data=_transform)
chain(_transform, _load)
chain(_create_table_if_not_exists, _load, _transform_in_target)

etlt()

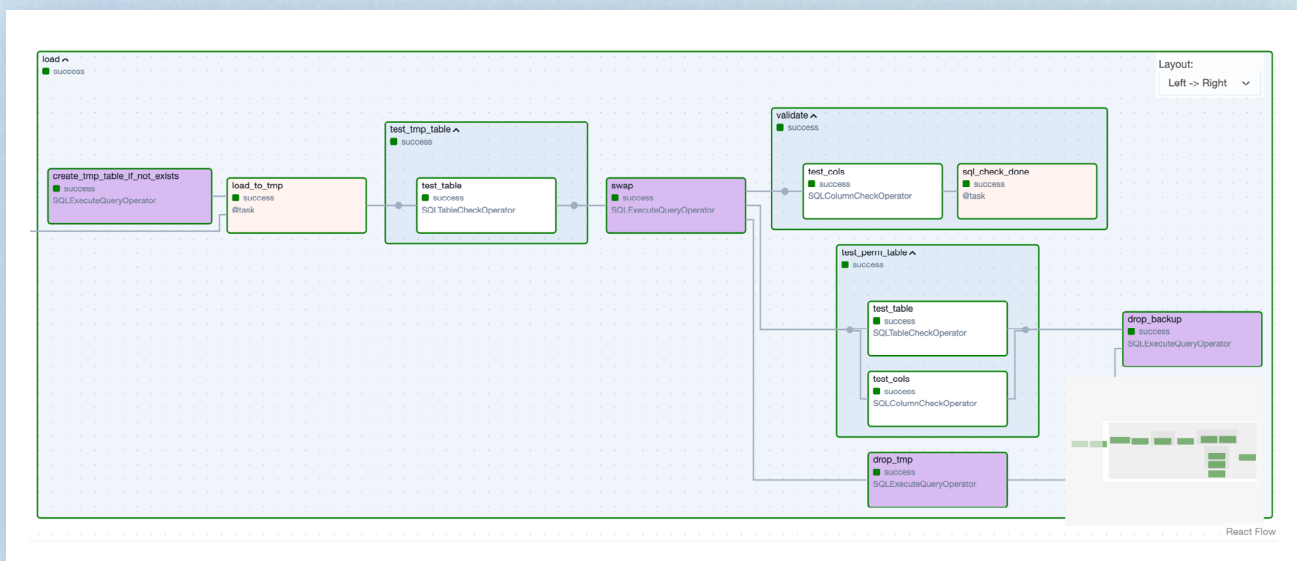
```

Advanced ETL DAG with data quality checks

This advanced DAG has the same extraction and transforming steps as the [ETL pattern DAG using XCom](#) but it loads the data using a temporary table and includes several data quality checks.

In summary this DAG:

1. Extracts weather data from the Open-Meteo API
2. Transforms the data
3. Loads the data into a temporary table in Postgres
4. Performs stopping data quality checks on the temporary table
5. Swaps the temporary table with the permanent table (and creates a backup of the old permanent table)
6. Drops the temporary table
7. Performs stopping data quality checks on the new permanent table
8. Drops the backup table
9. Performs additional warning data quality checks on the new permanent table



You can learn more about how to use the SQL check operators to write data quality checks [here](#). In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```
"""
```

```
## ETL with Data Quality Checks and Temporary Tables
```

```
This DAG will perform the following steps:
```

1. Extract weather data from the Open-Meteo API
2. Transform the data
3. Load the data into a temporary table in Postgres
4. Perform stopping data quality checks on the temporary table
5. Swap the temporary table with the permanent table (and creating a backup of the old permanent table)
6. Drop the temporary table
7. Perform stopping data quality checks on the new permanent table
8. Drop the backup table
9. Perform additional warning data quality checks on the new permanent table

```
"""
```

```
import os
```

```
from datetime import datetime, timedelta
```

```
from airflow.decorators import dag, task, task_group
```

```
from airflow.models.baseoperator import chain
```

```
from airflow.models.param import Param
```

```
from airflow.providers.common.sql.operators.sql import (
    SQLColumnCheckOperator,
    SQLExecuteQueryOperator,
    SQLTableCheckOperator,
)
```

```
DAG_ID = os.path.basename(__file__).replace(".py", "")
```

```
_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
```

```
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
```

```
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
```

```
_POSTGRES_TMP_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TMP", f"tmp_weather_data_{DAG_ID}"
)
```

```
_POSTGRES_BACKUP_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_BACKUP", f"backup_weather_data_{DAG_ID}"
)
```

```
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)
```

```

_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../include/sql/pattern_dags/{DAG_ID}"
)

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
    flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
    runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
    custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
        "postgres_conn_id": _POSTGRES_CONN_ID,
        "conn_id": _POSTGRES_CONN_ID,
    },
    tags=["Patterns", "ETL", "Postgres", "XCom"], # add tags in the UI
    params={
        "coordinates": Param({"latitude": 46.9481, "longitude": 7.4474}, type="object")
    }, # Airflow params can add interactive options on manual runs. See: https://www.
    astronomer.io/docs/learn/airflow-params
    template_searchpath=_SQL_DIR, # path to the SQL templates
)

def etl_dq_checks_tmp_tables():

    @task
    def extract(**context):
        """
        Extract data from the Open-Meteo API
        Returns:
            dict: The full API response
        """
        import requests

        url = os.getenv("WEATHER_API_URL")

        coordinates = context["params"]["coordinates"]

```

```

latitude = coordinates["latitude"]
longitude = coordinates["longitude"]

url = url.format(latitude=latitude, longitude=longitude)

response = requests.get(url)

return response.json()

@task
def transform_in_flight(api_response: dict, **context) → dict:
    """
    Transform the data
    Args:
        api_response (dict): The full API response
    Returns:
        dict: The transformed data
    """

    time = api_response["hourly"]["time"]
    dag_run_timestamp = context["ts"]

    transformed_data = {
        "temperature_2m": api_response["hourly"]["temperature_2m"],
        "relative_humidity_2m": api_response["hourly"]["relative_humidity_2m"],
        "precipitation_probability": api_response["hourly"][
            "precipitation_probability"
        ],
        "timestamp": time,
        "date": [
            datetime.strptime(x, "%Y-%m-%dT%H:%M").date().strftime("%Y-%m-%d")
            for x in time
        ],
        "day": [datetime.strptime(x, "%Y-%m-%dT%H:%M").day for x in time],
        "month": [datetime.strptime(x, "%Y-%m-%dT%H:%M").month for x in time],
        "year": [datetime.strptime(x, "%Y-%m-%dT%H:%M").year for x in time],
        "last_updated": [dag_run_timestamp for i in range(len(time))],
        "latitude": [api_response["latitude"] for i in range(len(time))],
        "longitude": [api_response["longitude"] for i in range(len(time))],
    }

    return transformed_data

_extract = extract()
_transform = transform_in_flight(_extract)

```



```

@task_group
def load():
    _create_tmp_table_if_not_exists = SQLExecuteQueryOperator(
        task_id="create_tmp_table_if_not_exists",
        database=_POSTGRES_DATABASE,
        sql="create_tmp_table_if_not_exists.sql",
        params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TMP_TABLE},
    )

@task
def load_to_tmp(transformed_data: dict):
    """
    Load the data to the destination without using a temporary CSV file.
    Args:
        transformed_data (dict): The transformed data
    """
    import csv
    import io

    from airflow.providers.postgres.hooks.postgres import PostgresHook

    hook = PostgresHook(postgres_conn_id=_POSTGRES_CONN_ID)

    csv_buffer = io.StringIO()
    writer = csv.writer(csv_buffer)
    writer.writerow(transformed_data.keys())
    rows = zip(*transformed_data.values())
    writer.writerows(rows)

    csv_buffer.seek(0)

    with open(f"{_SQL_DIR}/copy_insert.sql") as f:
        sql = f.read()
        sql = sql.replace("{schema}", _POSTGRES_SCHEMA)
        sql = sql.replace("{table}", _POSTGRES_TMP_TABLE)

    conn = hook.get_conn()
    cursor = conn.cursor()
    cursor.copy_expert(sql=sql, file=csv_buffer)
    conn.commit()
    cursor.close()
    conn.close()

_load_to_tmp = load_to_tmp(_transform)

```

```

@task_group
def test_tmp_table():

    SQLTableCheckOperator(
        task_id="test_table",
        retry_on_failure="True",
        table=f"{_POSTGRES_SCHEMA}.{_POSTGRES_TMP_TABLE}",
        checks={
            "row_count_check": {"check_statement": "COUNT(*) > 10"},
        },
    )

swap = SQLExecuteQueryOperator(
    task_id="swap",
    sql="swap.sql",
    params={
        "schema": _POSTGRES_SCHEMA,
        "tmp_table": _POSTGRES_TMP_TABLE,
        "perm_table": _POSTGRES_TRANSFORMED_TABLE,
    },
)

drop_tmp = SQLExecuteQueryOperator(
    task_id="drop_tmp",
    sql=f"DROP TABLE IF EXISTS {_POSTGRES_SCHEMA}.{_POSTGRES_TMP_TABLE};",
)

@task_group
def test_perm_table():

    SQLColumnCheckOperator(
        task_id="test_cols",
        retry_on_failure="True",
        table=f"{_POSTGRES_SCHEMA}.{_POSTGRES_TRANSFORMED_TABLE}",
        column_mapping={
            "month": {"min": {"geq_to": 1}, "max": {"leq_to": 12}},
            "day": {"min": {"geq_to": 1}, "max": {"leq_to": 31}},
        },
        accept_none="True",
    )

    SQLTableCheckOperator(
        task_id="test_table",
        retry_on_failure="True",
        table=f"{_POSTGRES_SCHEMA}.{_POSTGRES_TRANSFORMED_TABLE}",
        checks={
            "row_count_check": {"check_statement": "COUNT(*) > 10"},

```

```

        },
    )

drop_backup = SQLExecuteQueryOperator(
    task_id="drop_backup",
    sql=f"DROP TABLE IF EXISTS {_POSTGRES_SCHEMA}.{_POSTGRES_BACKUP_TABLE};",
)

chain(
    _create_tmp_table_if_not_exists,
    _load_to_tmp,
    test_tmp_table(),
    swap,
    [drop_tmp, test_perm_table()],
    drop_backup,
)

@task_group
def validate():
    test_cols = SQLColumnCheckOperator(
        task_id="test_cols",
        retry_on_failure="True",
        table=f"{_POSTGRES_SCHEMA}.{_POSTGRES_TRANSFORMED_TABLE}",
        column_mapping={
            "temperature_2m": {"min": {"geq_to": -10}, "max": {"leq_to": 50}},
        },
        accept_none="True",
    )

    @task(trigger_rule="all_done")
    def sql_check_done():
        return "Additional data quality checks are done!"

    test_cols >> sql_check_done()

    swap >> validate()

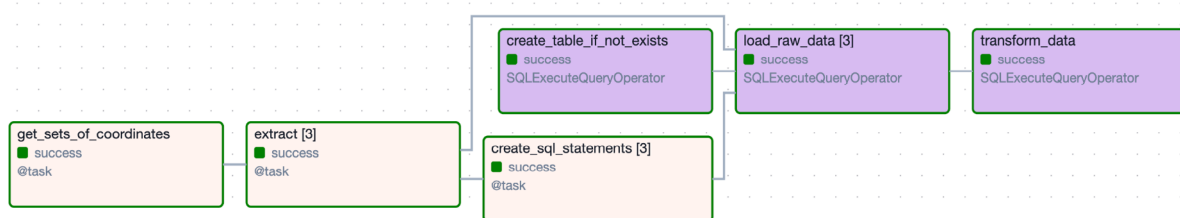
load()

etl_dq_checks_tmp_tables()

```

ELT DAG using Dynamic Task Mapping

The DAG below is similar to the [ELT pattern DAG using XCom](#), but instead of just fetching data from one API call including one set of coordinates, it uses [dynamic task mapping](#) to run 1 extraction and 1 loading tasks in parallel for each set of coordinates specified at runtime via the DAG configuration params.



The data is passed through XCom between the extraction and loading task, which means this setup would need a [custom XCom backend](#) in production.

In the companion GitHub repository you can find this DAG [here](#) and its supporting SQL scripts [here](#).

```

"""
## ELT DAG extracting data from 3 API calls in parallel using Dynamic Tasks

This DAG extracts weather data from the Open-Meteo API, loads it into a
postgres database and transforms it, using an ELT pattern.
It extracts data from 3 different locations in parallel using dynamic task mapping.
"""

import json
import os
from datetime import datetime, timedelta

from airflow.decorators import dag, task
from airflow.models.baseoperator import chain
from airflow.models.param import Param
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator

# ----- #
# DAG-level variables #
# ----- #

DAG_ID = os.path.basename(__file__).replace(".py", "")

_POSTGRES_CONN_ID = os.getenv("POSTGRES_CONN_ID", "postgres_default")
_POSTGRES_DATABASE = os.getenv("POSTGRES_DATABASE", "postgres")
_POSTGRES_SCHEMA = os.getenv("POSTGRES_SCHEMA", "public")
_POSTGRES_IN_TABLE = os.getenv("POSTGRES_WEATHER_TABLE_IN", f"in_weather_data_{DAG_ID}")
_POSTGRES_TRANSFORMED_TABLE = os.getenv(
    "POSTGRES_WEATHER_TABLE_TRANSFORMED", f"model_weather_data_{DAG_ID}"
)

_SQL_DIR = os.path.join(
    os.path.dirname(__file__), f"../..../include/sql/pattern_dags/{DAG_ID}"
)

# ----- #
# DAG definition #
# ----- #

```



```

@dag(
    dag_id=DAG_ID,
    start_date=datetime(2024, 9, 23), # date after which the DAG can be scheduled
    schedule="@daily", # see: https://www.astronomer.io/docs/learn/scheduling-in-air-
flow for options
    catchup=False, # see: https://www.astronomer.io/docs/learn/rerunning-dags#catchup
    max_active_runs=1, # maximum number of active DAG runs
    max_consecutive_failed_dag_runs=5, # auto-pauses the DAG after 5 consecutive failed
runs, experimental
    doc_md=__doc__, # add DAG Docs in the UI, see https://www.astronomer.io/docs/learn/
custom-airflow-ui-docs-tutorial
    default_args={
        "owner": "Astro", # owner of this DAG in the Airflow UI
        "retries": 3, # tasks retry 3 times before they fail
        "retry_delay": timedelta(seconds=30), # tasks wait 30s in between retries
    },
    tags=["Patterns", "ELT", "Postgres", "XCom"], # add tags in the UI
    params={
        "set_of_coordinates": Param(
            {
                "Location 1": {"latitude": 46.9480, "longitude": 7.4474},
                "Location 2": {"latitude": 38.4272, "longitude": 14.9524},
                "Location 3": {"latitude": 46.9480, "longitude": -2.9916},
            },
            type="object",
        )
    }, # Airflow params can add interactive options on manual runs. See: https://www.
astronomer.io/docs/learn/airflow-params
    template_searchpath=[_SQL_DIR], # path to the SQL templates
)

def elt_dtm():

    # ----- #
    # Task Definitions #
    # ----- #

    # the @task decorator turns any Python function into an Airflow task
    # any @task decorated function that is called inside the @dag decorated
    # function is automatically added to the DAG.
    # if one exists for your use case you can use traditional Airflow operators
    # and mix them with @task decorators. Checkout registry.astronomer.io for available
operators
    # see: https://www.astronomer.io/docs/learn/airflow-decorators for information about
@task
    # see: https://www.astronomer.io/docs/learn/what-is-an-operator for information

```

about traditional operators

```

_create_in_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_in_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_in_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_IN_TABLE},
)

_create_model_table_if_not_exists = SQLExecuteQueryOperator(
    task_id="create_model_table_if_not_exists",
    conn_id=_POSTGRES_CONN_ID,
    database=_POSTGRES_DATABASE,
    sql="create_model_table_if_not_exists.sql",
    params={"schema": _POSTGRES_SCHEMA, "table": _POSTGRES_TRANSFORMED_TABLE},
)

@task
def get_sets_of_coordinates(**context):
    """
    Get the set of coordinates from the params
    """
    coordinate_dict = context["params"]["set_of_coordinates"]
    list_of_coordinates = []
    for k, v in coordinate_dict.items():
        list_of_coordinates.append(v)

    return list_of_coordinates

_get_sets_of_coordinates = get_sets_of_coordinates()

@task(map_index_template="{{ my_custom_map_index }}")
def extract(coordinates: dict):
    """
    Extract data from the Open-Meteo API
    Returns:
        dict: The full API response
    """
    import requests

    url = os.getenv("WEATHER_API_URL")

```

```

latitude = coordinates["latitude"]
longitude = coordinates["longitude"]

url = url.format(latitude=latitude, longitude=longitude)

response = requests.get(url)

# optional custom map index (2.9+)
from airflow.operators.python import get_current_context

context = get_current_context()
context["my_custom_map_index"] = (
    "Coordinates: " + str(latitude) + ", " + str(longitude)
)

return response.json()

_extract = extract.expand(coordinates=_get_sets_of_coordinates)

@task
def create_sql_statements(data, schema, table):
    """
    Create SQL statements to load the data
    """
    data_json = json.dumps(data)

    return f"""
        INSERT INTO {schema}.{table} (raw_data)
        VALUES ('{data_json}'::jsonb);
    """

_create_sql_statements = create_sql_statements.partial(
    schema=_POSTGRES_SCHEMA, table=_POSTGRES_IN_TABLE
).expand(data=_extract)

_load = SQLExecuteQueryOperator.partial(
    task_id="load_raw_data",
    conn_id=_POSTGRES_CONN_ID,
).expand(sql=_create_sql_statements)

_transform = SQLExecuteQueryOperator(
    task_id="transform_data",
    conn_id=_POSTGRES_CONN_ID,
    sql="transform.sql",
    params={
        "schema": _POSTGRES_SCHEMA,
    },

```

```
        "in_table": _POSTGRES_IN_TABLE,  
        "out_table": _POSTGRES_TRANSFORMED_TABLE,  
    },  
)  
  
chain([_create_in_table_if_not_exists, _extract], _load, _transform)  
chain(_create_model_table_if_not_exists, _transform)  
  
elt_dtm()
```

ELT & ELT ON ASTRO

Simplify Your Workflows with Astronomer

When you use Astro to manage your ETL and ELT pipelines, you can unlock the full potential of your data and drive insightful business decisions with ease.

[Get Started Free](#)[Talk to an Expert](#)